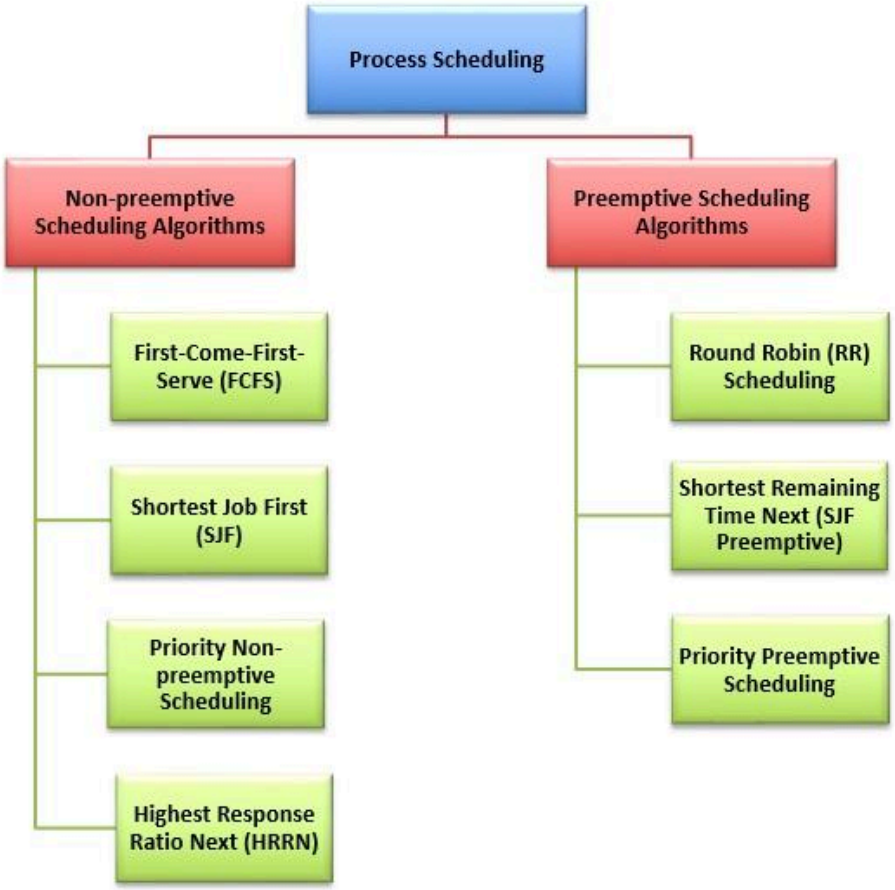
# CPU Scheduling Algorithms

In multiprogramming environments, process scheduling is performed by the CPU to decide the next process to be executed. This allows multiple CPU processes to exist simultaneously while optimizing utilization of system resources as well as the time of execution. A scheduling strategy defines a procedure to select one process among the processes waiting at the ready queue for execution.

## Categories of Process Scheduling

Process scheduling algorithms are broadly classified as **Preemptive Scheduling** and **Non-preemptive Scheduling**. The different types of preemptive and non-preemptive scheduling algorithms are represented in the following diagram –



# CPU Scheduling Algorithms

## Non-preemptive Scheduling

Non-preemptive scheduling algorithms refer to the class of CPU scheduling technique where once a process is allocated the CPU, it holds the CPU till the process gets terminated or is pushed to the waiting state. No process is interrupted until it runs to completion. The scheduler allocates another process to the CPU only after the currently allocated process terminates and relinquishes control on the CPU.

## Example of Non-preemptive Scheduling

- First-Come-First-Serve (FCFS) Scheduling
- Shortest Job First (SJF) Scheduling
- Priority Non-preemptive Scheduling
- Highest Response Ratio Next (HRRN) Scheduling

## Advantages of Non-preemptive Scheduling

- Non-preemptive scheduling methods are simpler to design and implement.
- These techniques need lesser resources and have lower overhead.
- They require a lesser number of context switches. So, the amount of time expended for scheduling is very less in comparison to execution of the processes.
- They are deterministic in nature and the scheduling outputs are easily predictable.

## Disadvantages of Non-preemptive Scheduling

- Non-preemptive scheduling cannot respond to dynamically changing systems.
- It may result in deadlock if there are processes in the system which are holding some resources while waiting for other resources that are held by other processes.

## CPU Scheduling Algorithms

- If the executing process erroneously enters an infinite loop, it may cause a system crash since no other process can interrupt the execution.

## Preemptive Scheduling

Preemptive scheduling algorithms fall under the category of process scheduling technique in which a running process can be interrupted by another process and sent to the ready queue even when it has not completed its entire execution in CPU.

An executing process may be pre-empted under the following situations –

- The first scenario is when the process has reached the limit of the time slot allotted to it. If the burst time of the process is greater than CPU cycle, it is placed back into the ready queue and will execute in the next chance.
- The second scenario is when a higher priority process enters the system. The process state of the executing process is saved in the process table and the higher priority process starts to execute.
- Another scenario is when a short process enters the system which uses the Shortest Remaining Time Next scheduling algorithm. If a longer process is executing, it is pre-empted to let the shorter process execute.

## Example of Preemptive Scheduling

- Round Robin (RR) Scheduling
- Shortest Remaining Time Next (SJF Preemptive)
- Priority Preemptive Scheduling

## Advantages of Preemptive Scheduling

- No process can monopolize the CPU.
- Fair scheduling can be applied.
- Preemptive scheduling improves the average response time.

## CPU Scheduling Algorithms

- They allow reconsideration of scheduling decisions after each pre-emption, thus making the scheduling decision more adaptive to change.
- Deadlocks can be avoided by preempting processes.

## Disadvantages of Preemptive Scheduling

- Preemptive scheduling requires frequent context switching which is time-consuming.
- This requires more memory since all context information for each process needs to be stored and updated frequently.
- Often a low priority process or a long process has to wait for a long time due to an influx of many higher priority or shorter processes in the system.
- The processes must be appropriately coded so that they can be switched at all times without information loss. This leads to complex processes.

## Difference Between Preemptive and Non-Preemptive Scheduling

The following table lists the key differences between preemptive and Non-pScheduling –

| Preemptive Scheduling | Non-Preemptive Scheduling |
|---|---|
| Resources are allocated according to the cycles for a limited time. | Resources are used and then held by the process until it gets terminated. |
| The process can be interrupted, even before the completion. | The process is not interrupted until its life cycle is complete. |

| | |
|---|---|
| Starvation may be caused, due to the insertion of priority process in the queue. | Starvation can occur when a process with large burst time occupies the system. |
| Maintaining queue and remaining time needs storage overhead. | No such overheads are required. |
| Fair scheduling can be applied where all the processes can get equal chance for CPU access | A process may monopolize the CPU. |
| Deadlocks can be easily avoided. | Deadlocks may occur. |

## FCFS Scheduling

FCFS is considered the simplest CPU-scheduling algorithm. In the FCFS algorithm, the process that requests the CPU first is allocated in the CPU first. The implementation of FCFS algorithm is managed with FIFO (First in first out) queue. FCFS scheduling is non-preemptive. Non-preemptive means, once the CPU has been allocated to a process, that process keeps the CPU until it executes a work or job or task and releases the CPU, either by requesting I/O.

### Salient Features of FCFS Algorithm

- FCFS is a non-preemptive scheduling algorithm.
- The process that arrives in the ready queue first is allocated to execute in the CPU first.
- Implementation is easy since it does not involve any complex algorithm.

- It does not require any prior knowledge about the processes. Also, if the run time behaviour of the processes changes dynamically, there is no impact on the FCFS scheduling algorithm.
- Though this assures fair scheduling, it may result in long waiting times for individual processes.

## Real Life Example Of FCFS Scheduling

As a real life example of FCFS scheduling a billing counter system of a shopping mall can be observed. The first person in the line gets the bill done first and then the next person gets the chance to get the bill and make payment and so on. If no priority is given to the VIP customers then the billing system will go on like this (means the first person (first task) in the line will get the bill first and after finishing (executing) the first customers payment the counter boy(CPU) will pay attention to other customers (separate tasks) as they are in the line). As FCFS is a non-preemptive type so no priority will be given to the random important tasks.

In CPU-scheduling problems some terms are used while solving the problems, so for conceptual purpose the terms are discussed as follows –

**Arrival time (AT)** – Arrival time is the time at which the process arrives in the ready queue.

**Burst time (BT) or CPU time of the process** – Burst time is the unit of time in which a particular process completes its execution.

**Completion time (CT)** – Completion time is the time at which the process has been terminated.

**Turn-around time (TAT)** – The total time from arrival time to completion time is known as turn-around time. TAT can be written as,

**Waiting time (WT)** Waiting time is the time at which the process waits for its allocation while the previous process is in the CPU for execution. WT is written as,

# CPU Scheduling Algorithms

**Turn-around time (TAT)** = Completion time (CT) − Arrival time (AT) or
**TAT** = Burst time (BT) + Waiting time (WT)

**Waiting time (WT)** = Turn-around time (TAT) − Burst time (BT)

**Response time (RT)** − Response time is the time at which CPU has been allocated to a particular process for the first time.

In case of non-preemptive scheduling, generally Waiting time and Response time is the same.

**Gantt chart** − Gantt chart is a visualization which helps to scheduling and managing particular tasks in a project. It is used while solving scheduling problems, for a concept of how the processes are being allocated in different algorithms.

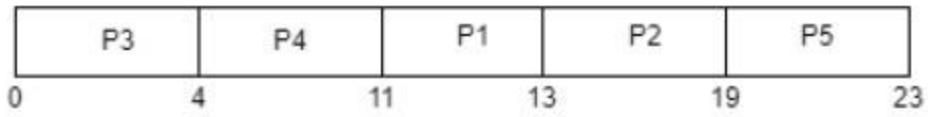
## Problem 1

Consider the given table below and find Completion time (CT), Turn-around time (TAT), Waiting time (WT), Response time (RT), Average Turn-around time and Average Waiting time.

| Process ID | Arrival time | Burst time |
|---|---|---|
| P1 | 2 | 2 |
| P2 | 5 | 6 |
| P3 | 0 | 4 |
| P4 | 0 | 7 |
| P5 | 7 | 4 |

## Solution

Gantt chart




For this problem CT, TAT, WT, RT is shown in the given table

| Process ID | Arrival time | Burst time | CT | TAT= CT-AT | WT= TAT-BT | RT |
|---|---|---|---|---|---|---|
| P1 | 2 | 2 | 13 | 13-2= 11 | 11-2= 9 | 9 |
| P2 | 5 | 6 | 19 | 19-5= 14 | 14-6= 8 | 8 |
| P3 | 0 | 4 | 4 | 4-0= 4 | 4-4= 0 | 0 |
| P4 | 0 | 7 | 11 | 11-0= 11 | 11-7= 4 | 4 |
| P5 | 7 | 4 | 23 | 23-7= 16 | 16-4= 12 | 12 |

**Average Waiting time** = (9+8+0+4+12)/5 = 33/5 = 6.6 time unit (time unit can be considered as milliseconds)

**Average Turn-around time** = (11+14+4+11+16)/5 = 56/5 = 11.2 time unit (time unit can be considered as milliseconds)

## Problem 2

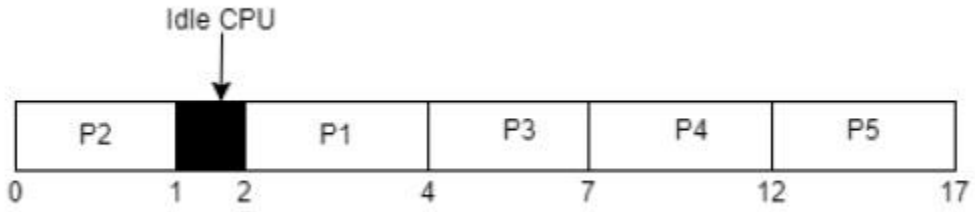
Consider the given table below and find Completion time (CT), Turn-around time (TAT), Waiting time (WT), Response time (RT), Average Turn-around time and Average Waiting time.

| Process ID | Arrival time | Burst time |
|---|---|---|
| P1 | 2 | 2 |
| P2 | 0 | 1 |

| | | |
|---|---|---|
| P3 | 2 | 3 |
| P4 | 3 | 5 |
| P5 | 4 | 5 |

## Solution

Gantt chart




For this problem CT, TAT, WT, RT is shown in the given table

| Process ID | Arrival time | Burst time | CT | TAT=CT-AT | WT=TAT-BT | RT |
|---|---|---|---|---|---|---|
| P1 | 2 | 2 | 4 | 4-2= 2 | 2-2= 0 | 0 |
| P2 | 0 | 1 | 1 | 1-0= 1 | 1-1= 0 | 0 |
| P3 | 2 | 3 | 7 | 7-2= 5 | 5-3= 2 | 2 |
| P4 | 3 | 5 | 12 | 12-3= 9 | 9-5= 4 | 4 |
| P5 | 4 | 5 | 17 | 17-4= 13 | 13-5= 8 | 8 |

**Average Waiting time** = (0+0+2+4+8)/5 = 14/5 = 2.8 time unit (time unit can be considered as milliseconds)

**Average Turn-around time** = (2+1+5+9+13)/5 = 30/5 = 6 time unit (time unit can be considered as milliseconds)

## CPU Scheduling Algorithms

*In idle (not-active) CPU period, no process is scheduled to be terminated so in this time it remains void for a little time.

## Advantages Of FCFS Scheduling

- It is an easy algorithm to implement since it does not include any complex way.
- Every task should be executed simultaneously as it follows the FIFO queue.
- FCFS does not give priority to any random important tasks first so it's a fair scheduling.

## Disadvantages Of FCFS Scheduling

- FCFS results in convoy effect which means if a process with higher burst time comes first in the ready queue then the processes with lower burst time may get blocked and that processes with lower burst time may not be able to get the CPU if the higher burst time task takes time forever.
- If a process with long burst time comes in the line first then the other short burst time process has to wait for a long time, so it is not as good as time-sharing systems.
- Since it is non-preemptive, it does not release the CPU before it completes its task execution completely.
- *Convoy effect and starvation sounds similar but there is slight difference, so it is advised to not treat these both terms as same words.

CPU Scheduling Algorithms

# SJF (SHORTEST JOB FIRST) Scheduling

In the Shortest Job First scheduling algorithm, the processes are scheduled in ascending order of their CPU burst times, i.e. the CPU is allocated to the process with the shortest execution time.

## Variants of SJF Scheduling

There are two variants of SJF scheduling –

### SJF non-preemptive scheduling

In the non-preemptive version, once a process is assigned to the CPU, it runs into completion. Here, the short term scheduler is invoked when a process completes its execution or when a new process(es) arrives in an empty ready queue.

### SJF preemptive scheduling

This is the preemptive version of SJF scheduling and is also referred to as Shortest Remaining Time First (SRTF) scheduling algorithm. Here, if a short process enters the ready queue while a longer process is executing, a process switch occurs by which the executing process is swapped out to the ready queue while the newly arrived shorter process starts to execute. Thus the short term scheduler is invoked either when a new process arrives in the system or an existing process completes its execution.

## Features of SJF Algorithm

- SJF allocates CPU to the process with the shortest execution time.
- In cases where two or more processes have the same burst time, arbitration is done among these processes on a first come first serve basis.
- There are both preemptive and non-premptive versions.
- It minimises the average waiting time of the processes.
- It may cause starvation of long processes if short processes continue to come into the system.

We can understand the workings of the two versions of this scheduling strategy through the aid of the following examples.

# CPU Scheduling Algorithms

## Examples of Non-Preemptive SJF Algorithm

### Example 1

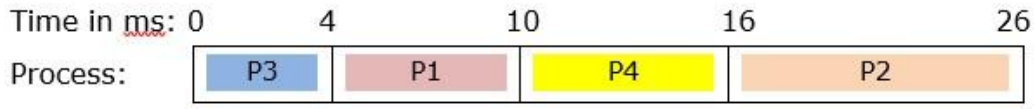
Suppose that we have a set of four processes that have arrived at the same time in the order P1, P2, P3 and P4. The burst time in milliseconds of each process is given by the following table –

| Process | CPU Burst Time in ms |
|---|---|
| P1 | 6 |
| P2 | 10 |
| P3 | 4 |
| P4 | 6 |

Let us draw the GANTT chart and find the average turnaround time and average waiting time using the non-preemptive SJF algorithm.

### GANTT Chart for the set of processes using SJF

Process P3 has the shortest burst time and so it executes first. Then we find that P1 and P4 have equal burst time of 6ms. Since P1 arrived before, the CPU is allocated to P1 and then to P4. Finally P2 executes. Thus the order of execution is P3, P1, P4, P2 and is given by the following GANTT chart –




Let us compute the average turnaround time and average waiting time from the above chart.

Average Turnaround Time

=Sum of Turnaround Time of each Process / Number of Processes

$$= (TAT_{P1} + TAT_{P2} + TAT_{P3} + TAT_{P4}) / 4$$

$$= (10 + 26 + 4 + 16) / 4 = 14 \text{ ms}$$

Average Waiting Time

$$= \text{Sum of Waiting Time of Each Process / Number of processes}$$

$$= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$$

$$= (4 + 16 + 0 + 10) / 4 = 7.5 \text{ ms}$$

## Example 2

In the previous example, we had assumed that all the processes had arrived at the same time, a situation which is practically impossible. Here, we consider the circumstances when the processes arrive at different times. Suppose we have set of four processes whose arrival times and CPU burst times are as follows –

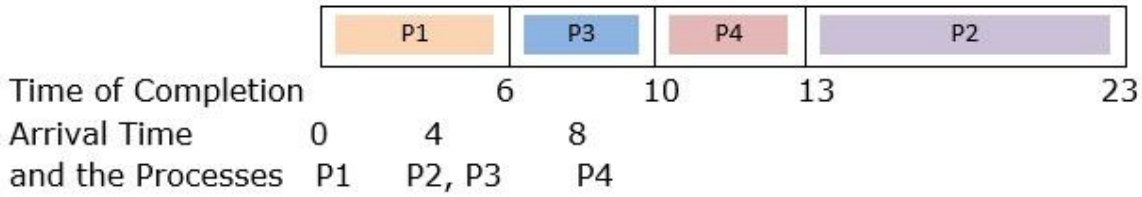
| Process | Arrival Time | CPU Burst Time |
|---|---|---|
| P1 | 0 | 6 |
| P2 | 4 | 10 |
| P3 | 4 | 4 |
| P4 | 8 | 3 |

Let us draw the GANTT chart and find the average turnaround time and average waiting time using non-preemptive SJF algorithm.

## GANTT Chart

While drawing the GANTT chart, we will consider which processes have arrived in the system when the scheduler is invoked. At 0ms, only P1 is there and so it is assigned to the CPU. P1 completes execution at 6ms and at that time P2 and P3 have arrived, but not P4. P3 is assigned to the CPU since it has the shortest burst time among current processes. P3 completes execution at time 10ms. By that time P4 has arrived and so the SJF algorithm is run on the processes P2 and P4. Hence, we find that the order of execution is P1, P3, P4, P2 as shown in the following GANTT chart –

# CPU Scheduling Algorithms




Let us calculate the turnaround time of each process and hence the average.

Turnaround Time of a process = Completion Time Arrival Time

$TAT_{P1} = CT_{P1} - AT_{P1} = 6 - 0 = 6$ ms

$TAT_{P2} = CT_{P2} - AT_{P2} = 23 - 4 = 19$ ms

$TAT_{P3} = CT_{P3} - AT_{P3} = 10 - 4 = 6$ ms

$TAT_{P4} = CT_{P4} - AT_{P4} = 13 - 8 = 5$ ms

Average Turnaround Time

=Sum of Turnaround Time of each Process/ Number of Processes

= (6 + 19+ 6 + 5) / 4 = 9 ms

The waiting time is given by the time that each process waits in the ready queue. For a non-preemptive scheduling algorithm, waiting time of each process can be simply calculated as –

Waiting Time of any process = Time of admission to CPU Arrival Time

$WT_{P1} = 0 - 0 = 0$ ms

$WT_{P2} = 13 - 4 = 9$ ms

$WT_{P3} = 6 - 4 = 2$ ms

$WT_{P4} = 10 - 8 = 2$ ms

Average Waiting Time

= Sum of Waiting Time of Each Process/ Number of processes

$= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$

$$= (0 + 9 + 2 + 2) / 4 = 3.25 \text{ ms}$$

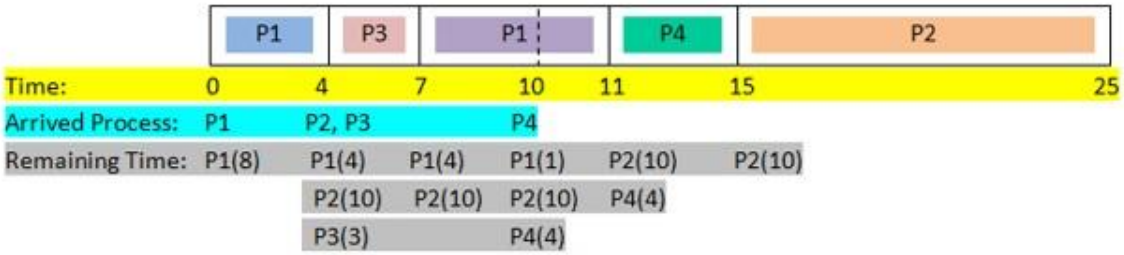
## Example of Preemptive SJF (SRTF) algorithm

Let us now perform preemptive SJF (SRTN) scheduling on the following processes, draw a GANTT chart and find the average turnaround time and average waiting time.

| Process | Arrival Time | CPU Burst Time |
|---|---|---|
| P1 | 0 | 8 |
| P2 | 4 | 10 |
| P3 | 4 | 3 |
| P4 | 10 | 4 |

## GANTT Chart

Since this is a preemptive scheduling algorithm, the scheduler is invoked when a process arrives and when it completes execution. The scheduler computes the remaining time of completion for each of the processes in the system and selects the process having the shortest remaining time left for execution.




Initially, only P1 arrives and so it is assigned to the CPU. At 4ms, P2 and P3 arrive. The scheduler computes the remaining time of the processes P1, P2 and P3 as 4ms, 10ms and 3ms. Since P3 has the shortest time, P1 is pre-empted by P3. P3 completes execution at 7ms and the scheduler is invoked. Among the processes in the system, P1 has the shortest time and

so it executes. At 10ms, P4 arrives and the scheduler again computes the remaining times left for each process. Since the remaining time of P1 is least, no process switch occurs and P1 continues to execute. In the similar fashion, the rest of the processes complete execution.

From the GANTT chart, we compute the average turnaround time and the average waiting time.

Average Turnaround Time

$$= \text{Sum of Turnaround Time of each Process/ Number of Processes}$$

$$= (TAT_{P1} + TAT_{P2} + TAT_{P3} + TAT_{P4}) / 4$$

$$= ((11 - 0) + (25 - 4) + (7 - 4) + (15 - 10)) / 4 = 10 \text{ ms}$$

Average Waiting Time

$$= \text{Sum of Waiting Time of Each Process/ Number of processes}$$

$$= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$$

$$= (3 + 11 + 0 + 1) / 4 = 3.75 \text{ ms}$$

## Advantages of SJF Algorithm

- In both preemptive and non-preemptive methods, the average waiting time is reduced substantially in SJF when compared to FCFS scheduling.
- SJF optimizes turnaround time to a considerable degree.
- If execution time of each process is estimated precisely, it promises maximum throughput.

## Disadvantages of SJF Algorithm

- In situations where there is an incoming stream of processes with short burst times, longer processes in the system may be waiting in the ready queue indefinitely leading to starvation.
- In preemptive SJF, i.e. SRTF, if all processes arrive at different times and at frequent intervals, the scheduler may be always working and

consequently the processor may be more engaged in process switching than actual execution of the processes.
- Correct estimation of the burst time is a complicated process. Since the effectiveness of the algorithm is entirely based upon the burst times, an erroneous calculation may cause inefficient scheduling.

## Conclusion

Shortest Job First scheduling can be termed as the optimal scheduling algorithm due to its theoretical best results. However, the implementation is much more complex and the execution is more unpredictable than First Come First Serve or Round Robin scheduling.

## Priority Scheduling

In systems where Priority Scheduling strategies are implemented, each process is assigned a priority value. Some systems follow the scheme that lower the priority value, higher the priority; while other systems follow the scheme of higher the priority value, higher the priority. The process with the highest priority value is selected for execution.

There are two variants of Priority scheduling –

### Non-preemptive Priority Scheduling

In the non-preemptive version, once a process is assigned to the CPU, it runs into completion. Here, the scheduler is invoked when a process completes its execution or when a new process(es) arrives in an empty ready queue. The scheduler chooses the process with highest priority for execution.

### Preemptive Priority Scheduling

In the preemptive version, if a high priority process enters the system while a lower priority process is executing, a process switch occurs by which the executing process is swapped out while the newly arrived higher priority process starts to execute. Thus the scheduler is invoked either when a new process arrives in the system or an existing process completes its execution.

Priority values can be also categorized into two forms –

**Static Priority**: In this system, once a priority value is assigned to a process, it remains constant as long as the process remains in the system.
**Dynamic Priority**: Here, the priority values change according to the nature of the process or the waiting time of the process in the system.

## Features of Priority Scheduling

The process with the highest priority is assigned to the CPU.
Non-preemptive priority scheduling that uses static priorities, are typically used in batch processes.
Preemptive priority scheduling that uses dynamic priority is used in most operating systems.
If two processes are of the same highest priority, then the scheduler arbitrates between them on a first come first serve basis.
Since most systems have some high priority system processes, priority scheduling finds its wide implementation, often in conjunction with other scheduling algorithms.

We can understand the workings Priority scheduling algorithm through the aid of the following examples –

## Examples of Non-Preemptive Priority Scheduling

### Example 1

Suppose that we have a set of four processes that have arrived at the same time in the order P1, P2, P3 and P4. The burst time in milliseconds and their priorities of the processes is given by the following table –

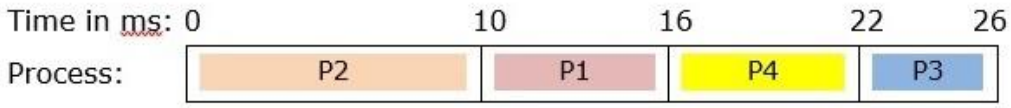
| Process | CPU Burst Time in ms | Priority |
|---|---|---|
| **P1** | 6 | 2 |
| **P2** | 10 | 1 |
| **P3** | 4 | 3 |
| **P4** | 6 | 2 |

# CPU Scheduling Algorithms

Considering that a lower priority value means higher priority, let us perform non-preemptive priority scheduling on it. We will draw the GANTT chart and find the average turnaround time and average waiting time.

## GANTT Chart

Process P2 has the highest priority and so it executes first. Then we find that both P1 and P4 have equal priority value of 2. Since P1 arrived before, the CPU is allocated to P1 and then to P4. Finally P3 executes. Thus the order of execution is P2, P1, P4, P3 and is given by the following




Let us compute the average turnaround time and average waiting time from the above chart.

Average Turnaround Time

= Sum of Turnaround Time of each Process/ Number of Processes

$= ( TAT_{P1} + TAT_{P2} + TAT_{P3} + TAT_{P4}) / 4$

$= ( 16 + 10 + 26 + 22) / 4 = 18.5$ ms

Average Waiting Time

= Sum of Waiting Time of Each Process/ Number of processes

$= ( WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$

$= ( 10 + 0 + 22 + 16) / 4 = 10.5$ ms

## Example 2

In this example, we consider a situation when the processes arrive at different times. Suppose we have set of four processes whose arrival times, CPU burst times and priorities are as follows –

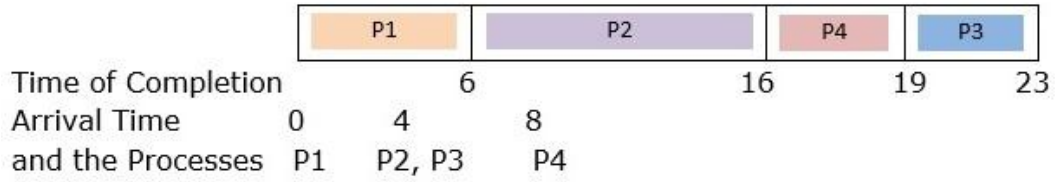
## CPU Scheduling Algorithms

| Process | Arrival Time | CPU Burst Time | Priority |
|---|---|---|---|
| **P1** | 0 | 6 | 2 |
| **P2** | 4 | 10 | 1 |
| **P3** | 4 | 4 | 2 |
| **P4** | 8 | 3 | 1 |

Let us draw the GANTT chart and find the average turnaround time and average waiting time using non-preemptive priority scheduling, considering lower priority value as higher priority.

## GANTT Chart

At 0ms, only P1 is there and so it is assigned to the CPU. P1 completes execution at 6ms and at that time P2 and P3 have arrived. P2 has higher priority and hence is assigned to the CPU. P2 completes execution at time 10ms. By that time P4 has arrived having priority value 1 and so it is assigned to the CPU. Once P4 completes execution, P3 is assigned to the CPU. So the order of execution is P1, P2, P4, P3 as shown in the following GANTT chart –




Let us calculate the turnaround time of each process and hence the average.

Turnaround Time of a process = Completion Time Arrival Time

$$TAT_{P1} = CT_{P1} - AT_{P1} = 6 - 0 = 6ms$$

$$TAT_{P2} = CT_{P2} - AT_{P2} = 16 - 4 = 12ms$$

$$TAT_{P3} = CT_{P3} - AT_{P3} = 23 - 4 = 19ms$$

$$TAT_{P4} = CT_{P4} - AT_{P4} = 19 - 8 = 11ms$$

## CPU Scheduling Algorithms

Average Turnaround Time

= Sum of Turnaround Time of each Process / Number of Processes

= ( 6 + 12+ 19 + 11) / 4 = 12 ms

The waiting time is given by the time that each process waits in the ready queue. For a non-preemptive scheduling algorithm, waiting time of each process can be simply calculated as −

Waiting Time of any process = Time of admission to CPU Arrival Time

$WT_{P1}$ = 0 - 0 = 0ms

$WT_{P2}$ = 6 - 4 = 2ms

$WT_{P3}$ = 19 - 4 = 15ms

$WT_{P4}$ = 16 - 8 = 8ms

Average Waiting Time

= Sum of Waiting Time of Each Process / Number of processes

= ($WT_{P1}$ + $WT_{P2}$ + $WT_{P3}$ + $WT_{P4}$) / 4

= ( 0 + 2 +15 + 8) / 4 = 6.25 ms

## Example of Preemptive Priority Scheduling

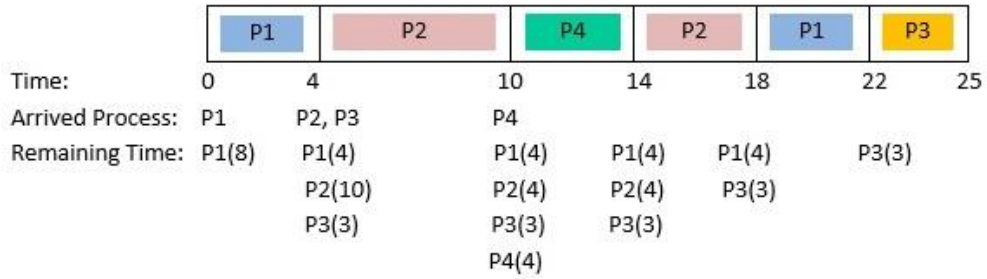
In preemptive priority scheduling, if a process arrives that has higher priority than the executing process, then the higher priority process pre-empts the lower priority process. Let us consider the following set of processes whose arrival times, burst times and priorities are given in the following table −

| Process | Arrival Time | CPU Burst Time | Priority |
|---|---|---|---|
| **P1** | 0 | 8 | 3 |
| **P2** | 4 | 10 | 2 |
| **P3** | 4 | 3 | 3 |

| | | | |
|---|---|---|---|
| **P4** | 10 | 4 | 1 |

## GANTT Chart

At time 0ms, P1 is the only process and so it starts executing. At time 4ms, P2 and P3 arrive. Since P2 has higher priority than P1, P2 preempts P1. At 10ms, P4 arrives which has higher priority than P2 and so pre-empts P2. P4 completes execution at 14ms and leaves. The processes in the system are P1, P2 and P3, among which P2 has highest priority and so is assigned to CPU. At 18ms, P2 completes execution and so P1 and P3 are the processes in the system. Since both processes are of same priority, the scheduler selects P1 by FCFS method. When P1 completes execution, finally P3 executes. The following GANTT chart shows the order of execution –




From the GANTT chart, we compute the average turnaround time and the average waiting time.

Average Turnaround Time

= Sum of Turnaround Time of each Process/ Number of Processes

$= (TAT_{P1} + TAT_{P2} + TAT_{P3} + TAT_{P4}) / 4$

= ((22-0) + (18-4) + (25-4) + (14-10)) / 4 = 15.25 ms

Average Waiting Time

= Sum of Waiting Time of Each Process/ Number of processes

$= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$

= (14 + 4 + 18 +0) / 4 = 7 ms

## Advantages of Priority Scheduling

- Implementation is simple since the scheduler doesn't require doing any prior calculations.
- Once the CPU defines the relative relevance (priorities) of the processes, the order of execution is easily predictable.
- Higher priority processes are almost served immediately.
- Priority scheduling is particularly helpful in systems that have a variety of processes each with their own needs.

## Disadvantages of Priority Scheduling

- In static priority systems, lower priority processes may need to wait indefinitely since the system is busy executing higher priority processes. This results in stagnation.
- Dynamic priority solves the stagnation problem. However, the added logic of dynamically updating priority values according to the system requires additional CPU cycles and thus increases load on the system.
- In non-preemptive priority scheduling, often a large process keeps shorter processes waiting for a long time.
- In preemptive priority scheduling, a low priority process may be repeatedly pre-empted by intermittent streams of high priority processes requiring frequent context switches.

## Conclusion

Priority scheduling algorithm paves way for more complex scheduling methods like multilevel queue scheduling. Assigning priorities to the processes helps the CPU to complete its important work first and leave the rest of the time for background processes. Priorities can be assigned to the processes as a function of their burst time, type of process, waiting time etc. and can thus incorporate the advantages of other basic scheduling algorithms.

# Round Robin (RR) Scheduling

This scheduling strategy derives its name from an age-old round-robin principle which advocated that all participants are entitled to equal share of assets or opportunities in a turn wise manner. In RR scheduling, each

process gets equal time slices (or time quanta) for which it executes in the CPU in turn wise manner. When a process gets its turn, it executes for the assigned time slice and then relinquishes the CPU for the next process in queue. If the process has burst time left, then it is sent to the end of the queue. Processes enter the queue on a first come first serve basis.

Round Robin scheduling is preemptive, which means that a running process can be interrupted by another process and sent to the ready queue even when it has not completed its entire execution in CPU. It is a preemptive version of First Come First Serve (FCFS) scheduling algorithm.

## Features of Round Robin Scheduling

- RR is a fair scheduling strategy where all processes get equal share to execute in turn wise manner.
- It is a preemptive scheduling method where an executing process must give up its control of the CPU once its time quantum expires.
- Through RR scheduling strategy, none of the processes go into starvation.
- It is widely used for its simple working principle.
- The performance of RR scheduling is vastly dependent on the chosen time quantum.

## Working Principle of Round Robin Scheduling

- Any new process that arrives the system is inserted at the end of the ready queue in FCFS manner.
- The first process in the queue is removed and assigned to the CPU.
- If the required burst time is less than or equal to the time quantum, the process runs to completion. The scheduler is invoked when the process completes executing to let in the next process in the ready queue to the CPU.
- If the required burst time is more than the time quantum, the process executes up to the allotted time quantum. Then its PCB (process control block) status is updated and it is added to the end of the queue. Context switch occurs and the next process in the ready queue is assigned to the CPU.
- The above steps are repeated until there are no more processes in the ready queue.

## CPU Scheduling Algorithms

We can understand the workings RR scheduling algorithm through the aid of the following example.

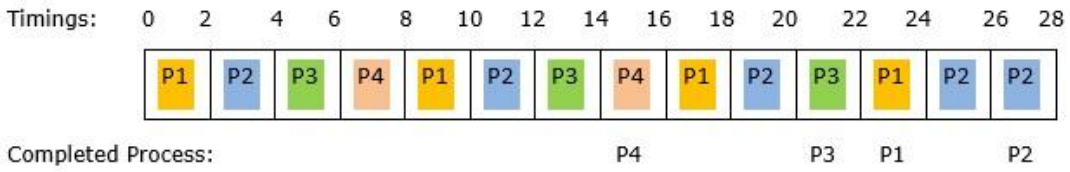
## Example of Round Robin Scheduling

Let us consider a system that has four processes which have arrived at the same time in the order P1, P2, P3 and P4. The burst time in milliseconds of each process is given by the following table –

| Process | CPU Burst Times in ms |
|---|---|
| P1 | 8 |
| P2 | 10 |
| P3 | 6 |
| P4 | 4 |

Let us consider time quantum of 2ms and perform RR scheduling on this. We will draw GANTT chart and find the average turnaround time and average waiting time.

## GANTT Chart with time quantum of 2ms




## Average Turnaround Time

Average TAT = Sum of Turnaround Time of each Process **/** Number of Processes

= ( 24 + 28 + 22+ 16) / 4 = 22.5 ms

## CPU Scheduling Algorithms

In order to calculate the waiting time of each process, we multiply the time quantum with the number of time slices the process was waiting in the ready queue.

## Average Waiting Time

Average WT = Sum of Waiting Time of Each Process **/** Number of processes

= ( 8*2 + 9*2+ 8*2+ 6*2) / 4 = 15.5 ms

## Advantages of Round Robin Scheduling

- Round Robin scheduling is the most a fair scheduling algorithm whereby all processes are given equal time quantum for execution.
- Starvation of any process caused by indefinite waiting in ready queue is totally eliminated in RR scheduling.
- It does not require any complicated method to calculate the CPU burst time of each process prior to scheduling.
- It is pretty simple to implement and so finds application in a wide range of situations.
- Convoy effect does not occur in RR scheduling as in First Come First Serve CPU (FCFS) scheduling.

## Disadvantages of Round Robin Scheduling

- The performance of Round Robin scheduling is highly dependent upon the chosen time quantum. This requires prudent analysis before implementation, failing which required results are not received.
- If the chosen time quantum is very large, most of the processes with complete within the burst time. In effect, RR scheduling will act as FCFS scheduling. Thus, all the limitations of FCFS will come into the system.
- If the chosen time quantum is too small, the CPU will be very busy in context switching, i.e. swapping in swapping out processes to and from the CPU and memory. This would reduce the throughput of the system since more time will be expended in context switching rather than actual execution of the processes.
- RR scheduling does not give any scope to assign priorities to processes. So, system processes which need high priority gets the

same preference as background processes. This may often hamper the overall performance of a system.

## Conclusion

Round Robin scheduling, if properly implemented, provide the simplest and the best solutions to scheduling problems. A number of variations of RR scheduling are being researched upon and implemented, in order to avoid the disadvantages of this algorithm. One variant that helps to provide near perfect time quantum is Dynamic Time Quantum Scheduling. Here, the time quantum dynamically varies according to the behaviour of the processes in the system. Another variant, the Selfish Round Robin scheduling assigns priorities to processes and provides more CPU slices to higher priority processes.

## Difference between Turn Around Time (TAT) and Waiting Time (WT) in CPU Scheduling

The following table highlights the major differences between TAT and WT in CPU scheduling –

| S.N. | Criteria | Turn Around Time (TAT) | Waiting Time (WT) |
|---|---|---|---|
| 1 | Definition | The total time taken for a process to complete its execution | The total time a process spends waiting in the ready queue |
| 2 | Calculation | Completion time Arrival time | Start time of execution Arrival time |
| 3 | Inclusion | Includes both waiting time and execution time | Focuses solely on waiting time in the ready queue |

| | | | |
|---|---|---|---|
| 4 | Evaluation | Assesses the overall efficiency and performance of a scheduling algorithm | Reflects waiting experience and process responsiveness |
| 5 | Impact | Provides a comprehensive measure of process turnaround time | Directly affects system responsiveness and reduces idle time |
| 6 | Importance | Critical for evaluating the efficiency and performance of a scheduling algorithm | Crucial for improving system responsiveness and reducing waiting time |
| 7 | Optimization | Lower TAT indicates faster execution and better system performance | Minimizing WT improves system responsiveness and resource utilization |
| 8 | Measurement | Measured from the process arrival to completion | Measured from the process arrival to the start of execution |
| 9 | Purpose | Evaluates the efficiency of a scheduling algorithm as a whole | Focuses on individual process waiting times in the ready queue |

# CPU Scheduling Algorithms

In operating systems, **starvation** and **aging** are two important concepts related to process scheduling and resource allocation. Starvation is a situation where a process is denied access to resources it needs for execution, and aging is a technique used to prevent starvation. In this chapter, we will discuss both concepts in detail.

## Starvation in Operating System

**Starvation** is a situation in which a process is not able to gain access to the resources it needs for execution. This happens when the CPU scheduling algorithm prefers other higher priority processes over the lower priority ones. As a result, some processes may need to wait indefinitely while other processes are being executed. Starvation is also known as **indefinite blocking**.

## Example of Starvation

Consider a system that uses **priority scheduling** for CPU allocation. In this system, processes with higher priority are executed first. For example, assume that there are three processes P1, P2, and P3 with priorities 3, 2, and 1 respectively (higher number indicates higher priority). Following is the sequence of events −

```
Process  Priority

P1       3 (high priority)

P2       2

P3       1


Process arrives in the order:

P1, P2, P3, P2, P1, P2, P2, P1, P3, P1, P2, ...


CPU allocation based on priority:

P1 -> P2 -> P2 -> P1 -> P2 -> P2 -> P2 -> P1 -> P1 -> P2 -> ...
```

```
Result: P3 never gets executed
```

In this example, P3 got starved because it has the lowest priority. As long as there are higher priority processes (P1 and P2) in the queue, P3 will never get a chance to execute.

## Causes of Starvation

The main causes of starvation in operating systems are –

**Priority Scheduling** – In priority scheduling algorithms, processes with higher priority are executed first. If there are always higher priority processes in the queue, lower priority processes may never get executed.

**Resource Allocation** – If resources are allocated in a way that some processes are always denied access to the resources they need, those processes may starve.

**Deadlock** – In some cases, deadlock can lead to starvation of multiple processes.

# Aging in Operating System

**Aging** is a technique used to prevent starvation in operating systems. The idea behind aging is to gradually increase the priority of a process that are waiting in the queue for a long time. This way, even lower priority processes will eventually get a chance to execute.

## Example of Aging

Continuing from the previous example, let's apply aging to the processes. Assume that for every time unit a process waits in the queue, its priority is increased by 1. The updated priorities of the processes after some time would be –

```
Process arrives in the order:

P1, P2, P3, P2, P1, P2, P2, P1, P3, P1, P2, ...
```

```
CPU allocation based on priority with aging:

Allocation                          Priority Now

-> Allocate P1 (priority 3)         {P1: 3, P2: 2, P3: 1}

-> Allocate P2 (priority 2)         {P1: 3, P2: 3, P3: 2}

-> Allocate P3 (priority 3)         {P1: 3, P2: 3, P3: 3}
```

```
Result: No starvation, all processes get executed
```

In this example, after applying aging, P3's priority eventually increased to match that of P1 and P2. As a result, P3 got a chance to execute.

## Conclusion

**Starvation** is a situation where a process is waiting for infinite time due to resource allocation policies. To prevent starvation, operating systems use **aging** techniques, here the priority of waiting processes is gradually increased over time. Meaning the older processes will have high priority over time. This ensures that all processes eventually get a chance to execute.